



Engs 28

Embedded Systems

Day 3x

Review

On Tuesday:

- Introduction to oscilloscope measurements with AD2 / Waveforms.
- How to interface a pushbutton switch to an STM32C0 I/O pin.

Today:

- Back to the oscilloscope
- More about switches and inputs.

Need a C refresher? Check out

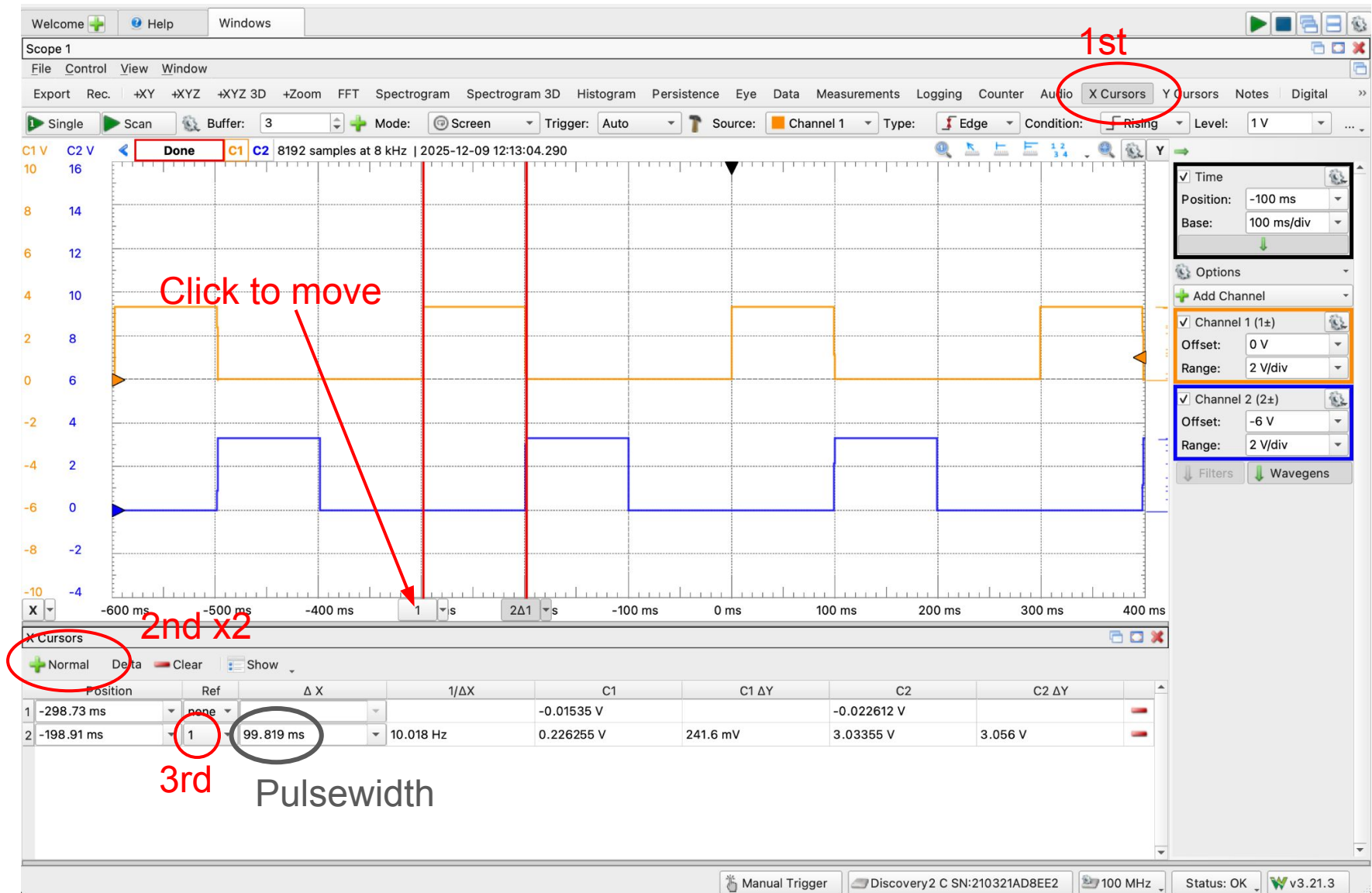
<https://engs20book.thayer.dartmouth.edu/ENG28Guide>

(linked under "Resources" on Canvas)

Use “flying leads” to connect M2K to your breadboard: “–” leads to ground, “+” leads to the LED anodes.



Measure Pulse Width with Cursors



One more experiment

Edit **blinkySIM.c**:

1. Change the blink rate as follows and note the brightness of the LEDs and the waveform on Scopy

```
while(1) {  
    GPIOA->ODR |= LED_ALL;           // 3. turn LEDs on  
    delay_ms(1);                     // kill some time  
  
    GPIOA->ODR &= ~LED_ALL;          // 4. turn LEDs  
    delay_ms(10);                    // kill some time  
}
```

2. Now swap the lengths of on and off time; again note the brightness and the waveform.

```
while(1) {  
    GPIOA->ODR |= LED_ALL;           // 3. turn LEDs on  
    delay_ms(10);                   // kill some time  
  
    GPIOA->ODR &= ~LED_ALL;          // 4. turn LEDs  
    delay_ms(1);                    // kill some time  
}
```

The next slide shows the oscilloscope traces (so indeed the timing is as desired.)

Discuss at your table why the LED brightness changes. How could this be useful?

```

while(1) {
    GPIOA->ODR |= LED_ALL;      // 3. turn LEDs on
    delay_ms(1);                // kill some time

    GPIOA->ODR &= ~LED_ALL;      // 4. turn LEDs
                                // kill some time
    delay_ms(10);               // kill some time
}

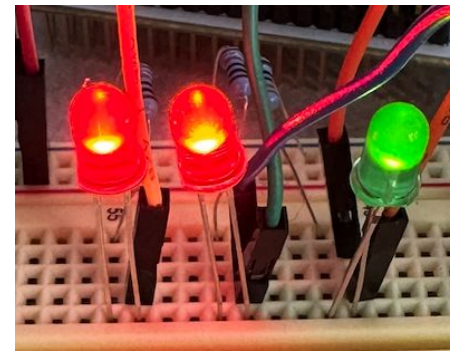
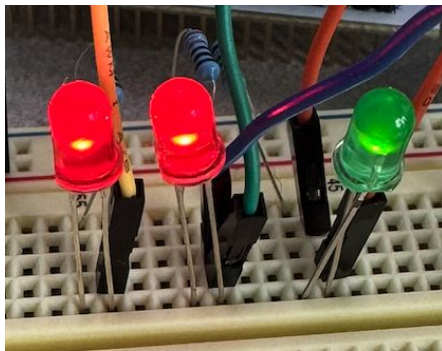
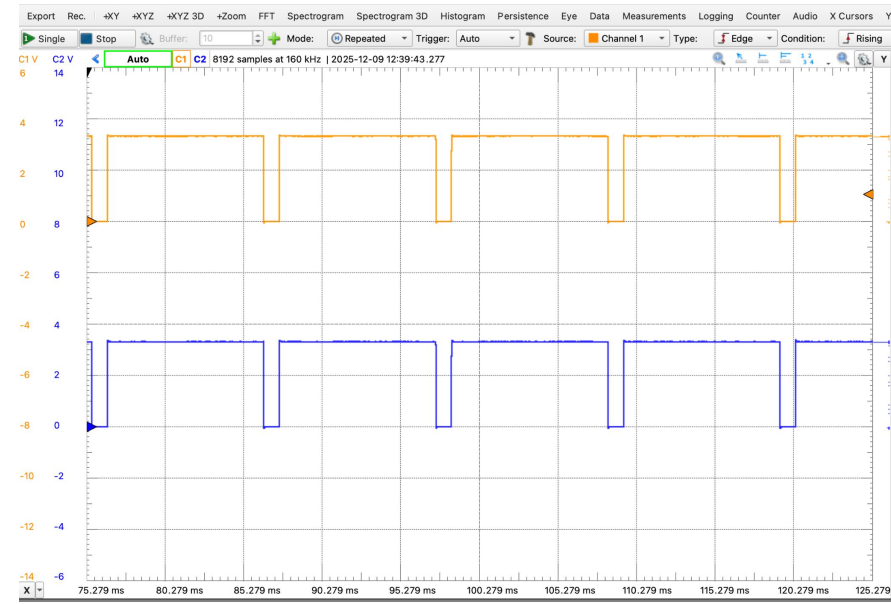
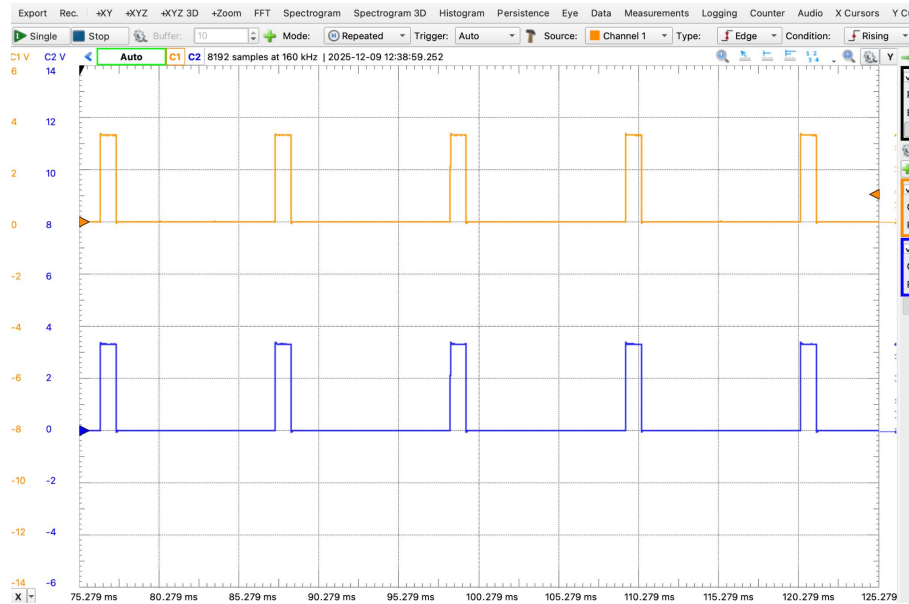
```

```

while(1) {
    GPIOA->ODR |= LED_ALL;      // 3. turn LEDs on
    delay_ms(10);              // kill some time

    GPIOA->ODR &= ~LED_ALL;      // 4. turn LEDs
                                // kill some time
    delay_ms(1);               // kill some time
}

```

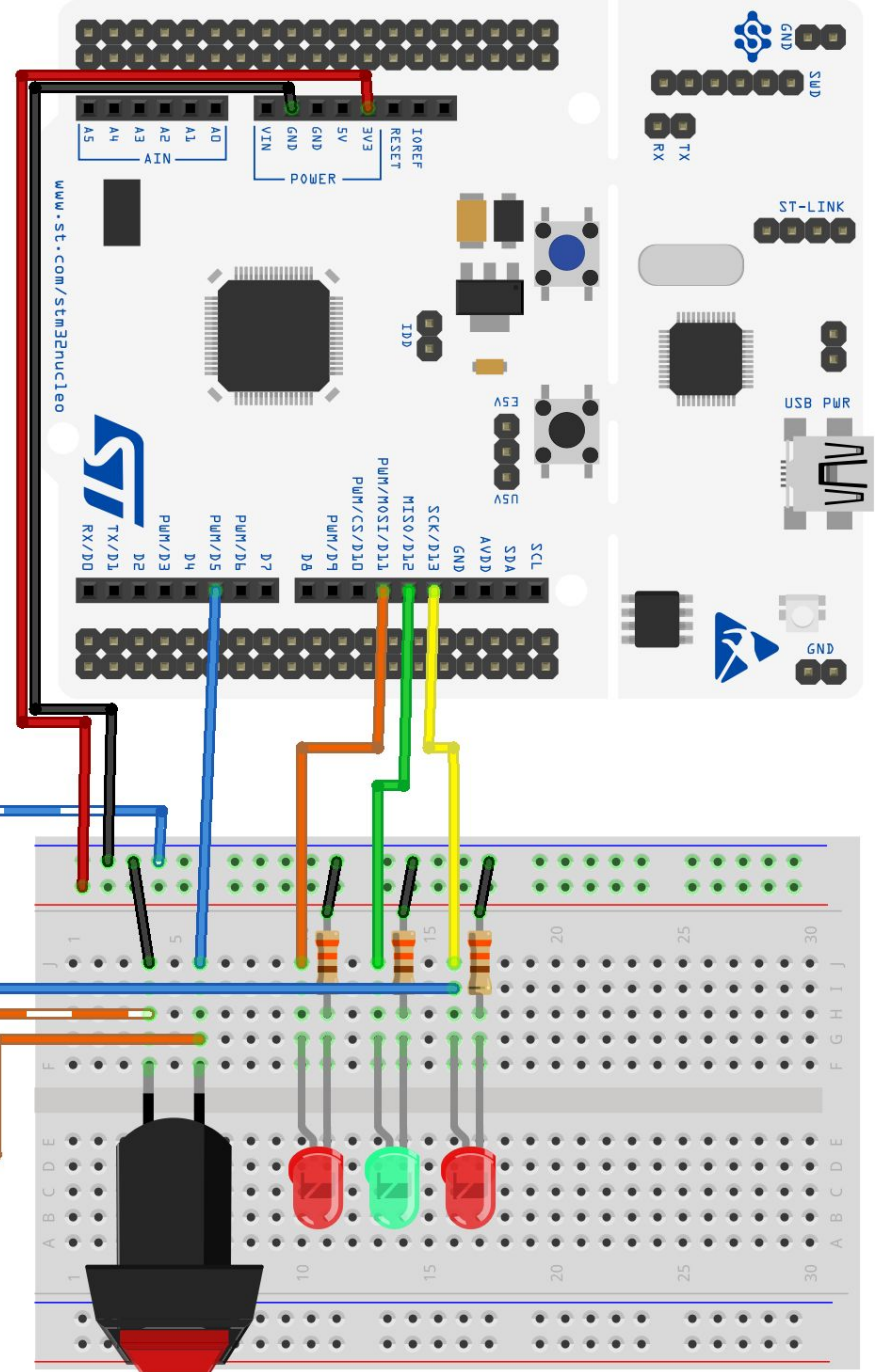
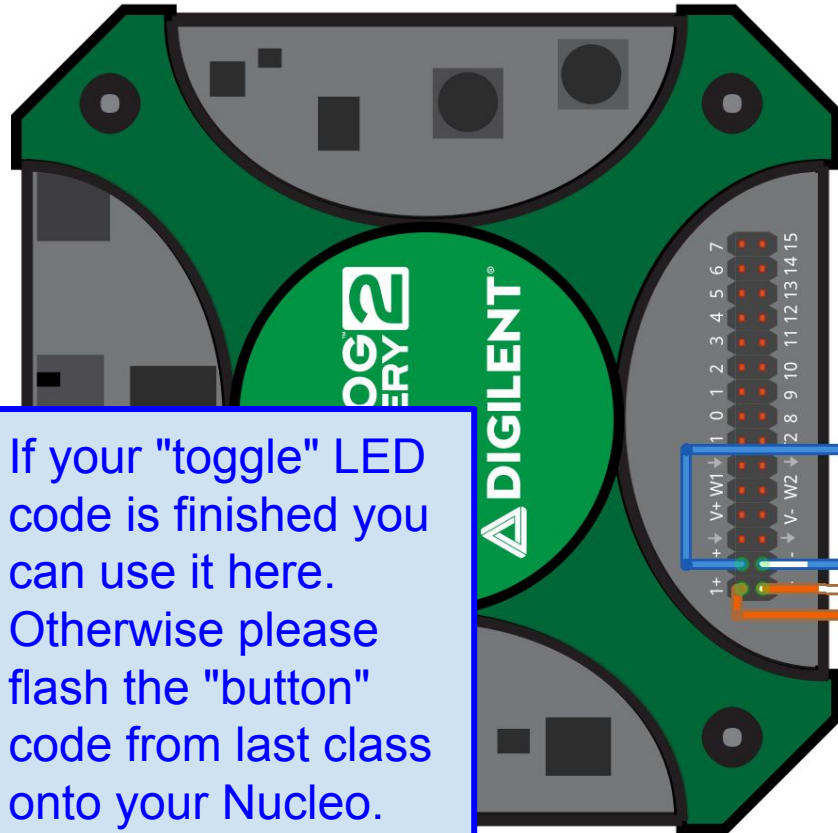


Back to our Button

Use oscilloscope to observe button behavior (wiring)

Channel 1: Button on D5 (PB4)

Channel 2: LED on D13 (PA5)



Load button.c onto your Nucleo (unless you already finished the toggle code).

```
#include "ES28.h"                // All the port definitions are here

#define GPIOAEN      (1U<<0)    // Clock access to GPIOA (LED)
#define GPIOBEN      (1U<<1)    // Clock access to GPIOB (button)

#define LED_PIN      (1U<<5)    // LED on PA5      (D13)
#define BUTTON_PIN   (1U<<4)    // Button on PB4 (D5)

int main(void) {
    RCC->IOPENR |= (GPIOAEN | GPIOBEN); // Enable clock access to GPIOA and B

    GPIOA->MODER |= (1U<<10);           // Configure PA5 as output pin
    GPIOA->MODER &= ~(1U<<11);

    GPIOB->MODER &= ~(1U<<8);           // Configure PB4 as input pin
    GPIOB->MODER &= ~(1U<<9);           // (not necessary since this is default reset value)

    GPIOB->PUPDR |= (1U<<8);            // enable pull-up for PB4
    GPIOB->PUPDR &= ~(1U<<9);

    delay_ms(50);

    while(1) {
        // Read the pin and control the LED
        if ((GPIOB->IDR & BUTTON_PIN) == 0) { // button input is low-true
            GPIOA->ODR |= LED_PIN;             // turn LED on
        } else {
            GPIOA->ODR &= ~LED_PIN;            // turn LED off
        }
    }
    return 0;                                // never reached
}
```

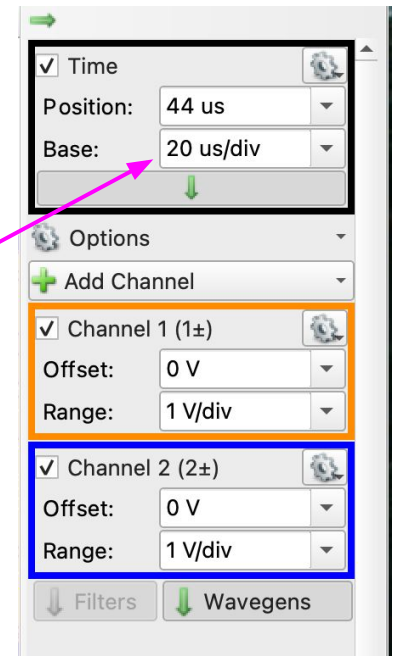
Use scope to observe button (Waveforms setup)



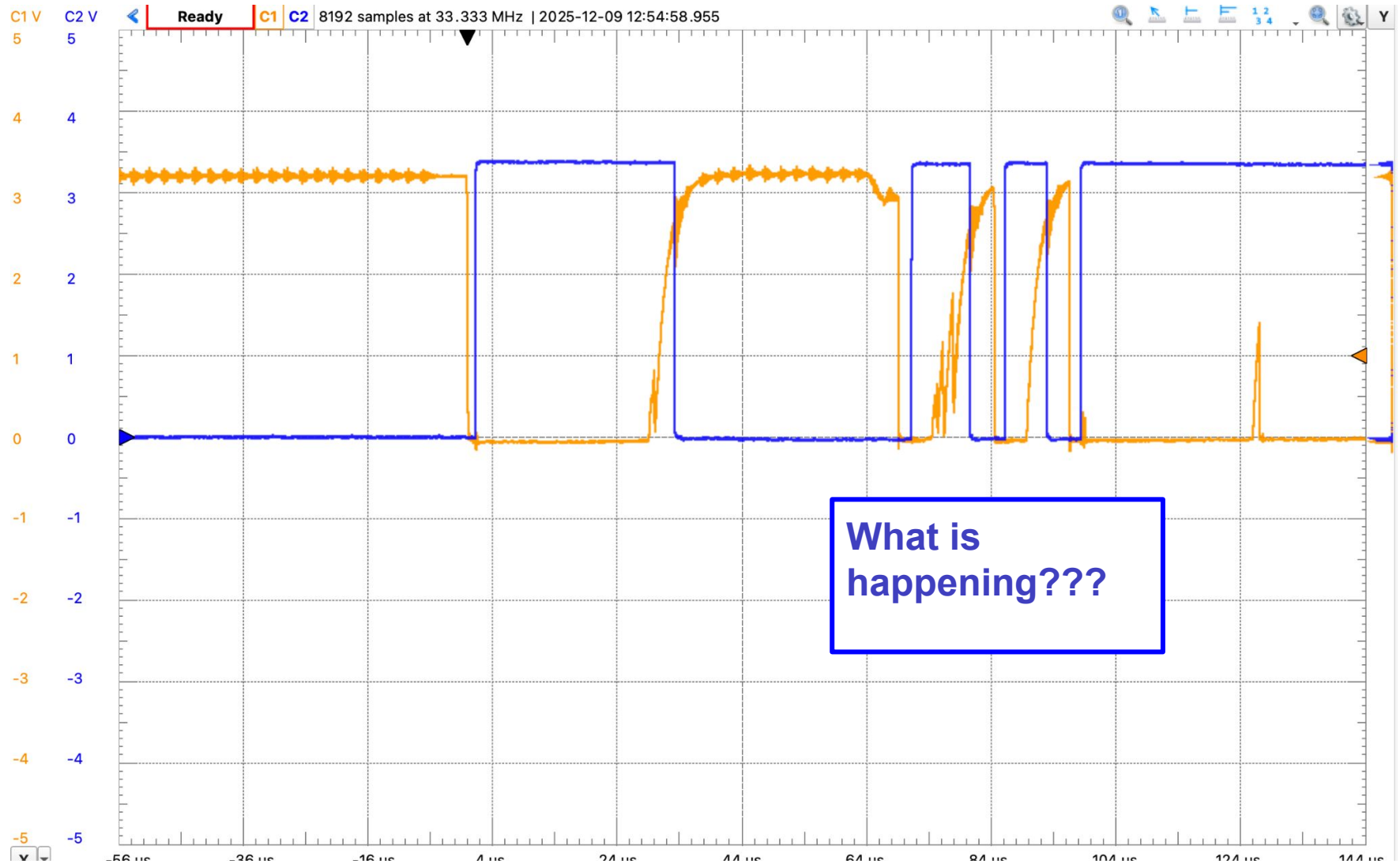
May need to switch this to 'auto', depending on the software version.

The 'scope will begin an acquisition when **Channel 1** (button signal) **falls** below **1V**.

20us/div is a good start. Might need to change depending on your button



Click "Single", then press the button once to trigger.



How to “debounce” a pushbutton

The switch contacts “bounce” for a while (here, about $120\mu\text{s}$, can be up to 10ms , depending on mechanical structure of switch) before making a solid connection.

Debouncing consists of delaying the input to the pin until the bouncing stops.

- Analog (simple circuit)
- Software (e.g., `delay_ms`)
- Digital timer (Engs 31 solution)

Hardware or software?

Hardware (RC circuit, digital timer)

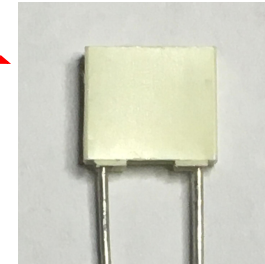
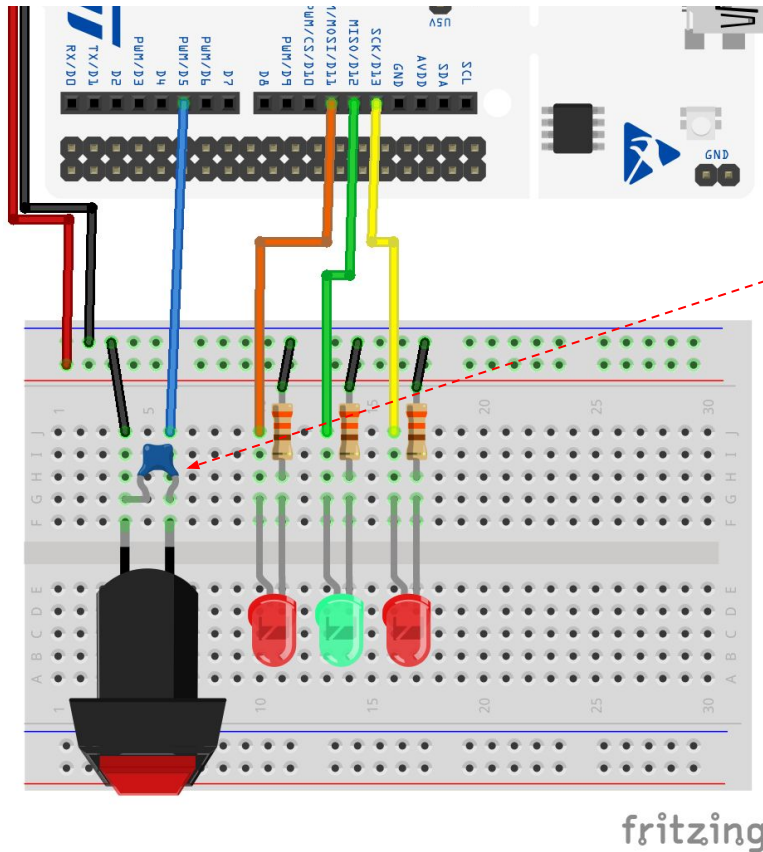
- (+) Doesn't tie up processor cycles.
- (−) Requires another (tiny) component on circuit board, additional wiring
- (−) Hard to revise for a different switch; design with tolerance.

Software (software delay)

- (+) Flexible. Just edit code and reflash if the switch is changed.
- (+) Simple. Doesn't require additional components on the circuit board.
- (−) Wastes processor cycles while waiting for the bouncing to stop.

Let's try the analog solution.

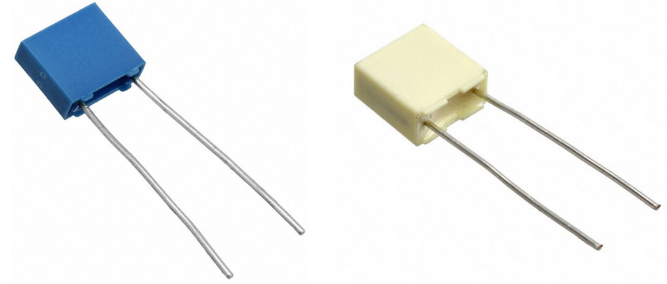
Connect a **0.1 μ F capacitor** across the switch and note the effect (we'll see why it works soon).



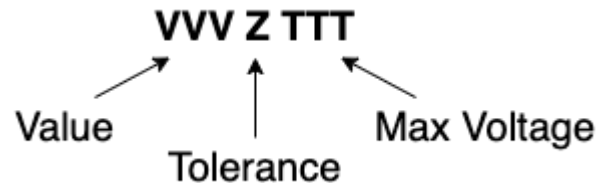
How do you know which of the capacitors is 0.1 μ F?

Why does this work?

Reading Capacitor Codes



Our Metal Film Capacitors have 3 fields in their marking:



Value is 2 digits and a decimal point.

The decimal point can be an n, a p, or a dot

The default values (dot) are in microfarads.

P means Pico

N mean Nano.

Tolerance is a single letter.

J means +/-5%

K means +/-10%

Replace the letter with a decimal point and move the letter to the end:

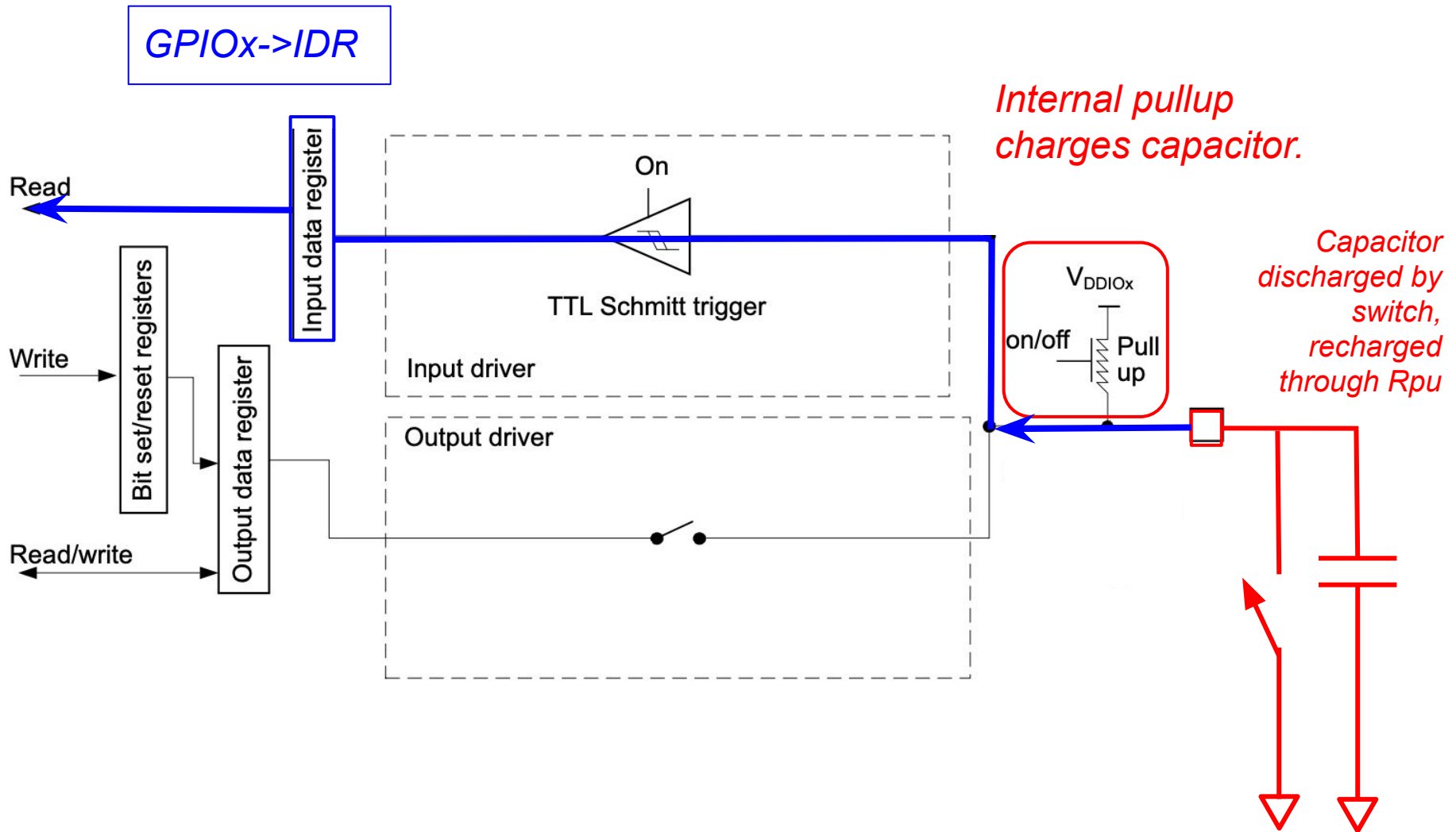
22nJ100 => 22.0nF +/- 5% Max 100V

.15J63 => 0.15uF +/- 5% Max 63V

2n2K100 => 2.2nF +/-10% Max 100V

p22J63 => 0.22pF +/- 5% Max 63V

How to wire a pushbutton



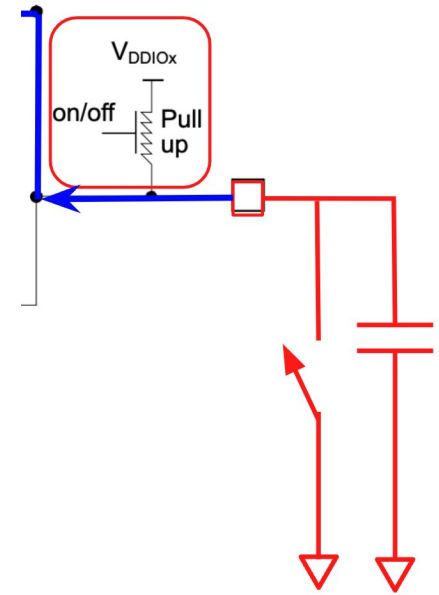
How does it work?

Remember from Phys 14: a capacitor stores electric charge.

1. When the switch is open, capacitor is kept charged by internal pullup R_{pu} .
2. When the switch is closed, capacitor discharges.
3. If the switch bounces open, capacitor recharges through pullup. Recharge time proportional to $R_{pu}C$.
4. Discharge-charge continues until bouncing ceases.

Choose a value of C so that $R_{pu}C$ is sufficiently long to prevent the bounce from rising to V_{IH} .

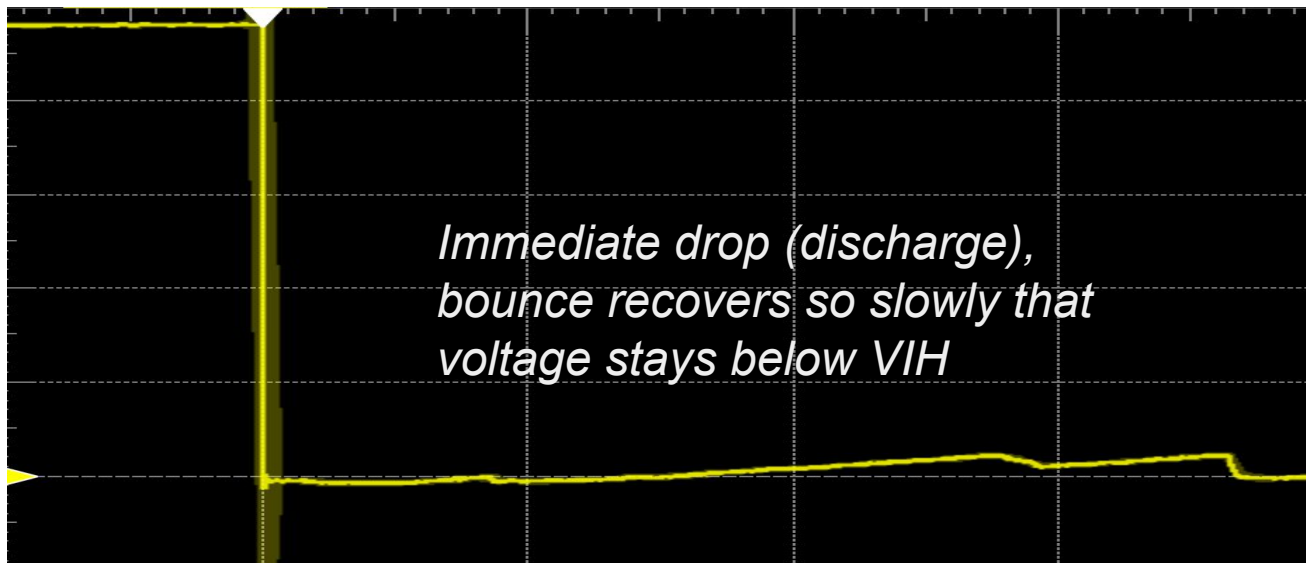
Quantitative design in Lab 2.



Results



$C=0.01\mu\text{F}$

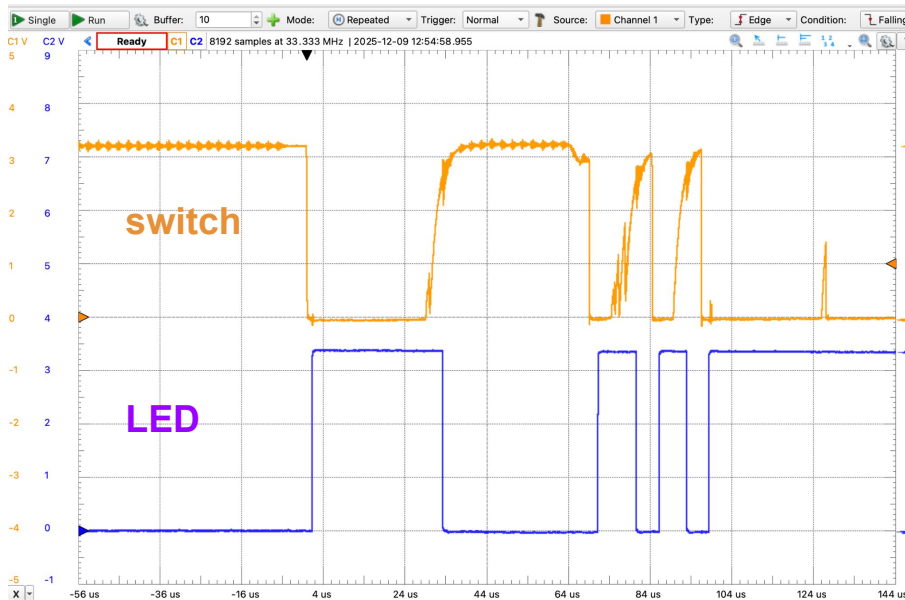


$C=0.1\mu\text{F}$

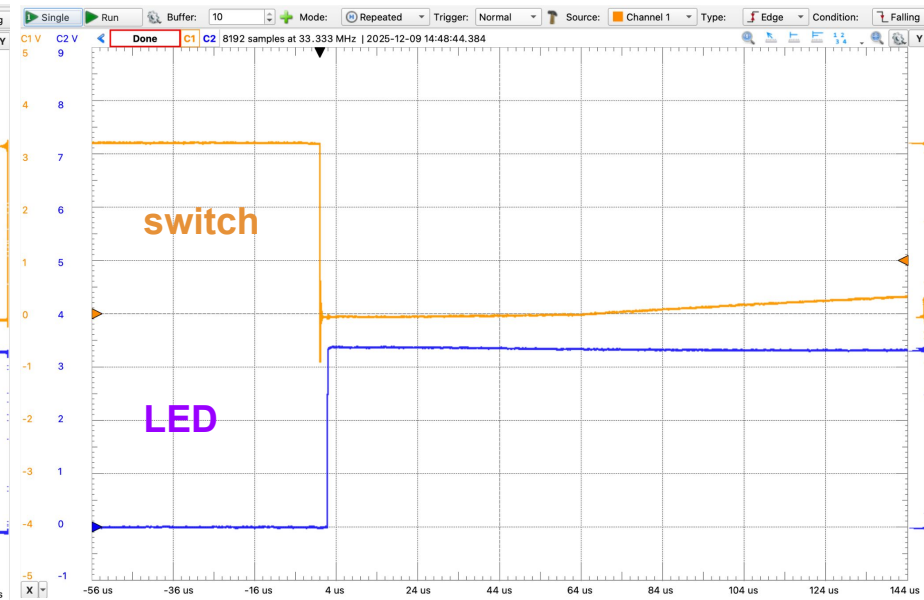
Use oscilloscope to observe glitches

Also called “switch bounce”

Without capacitor



With capacitor

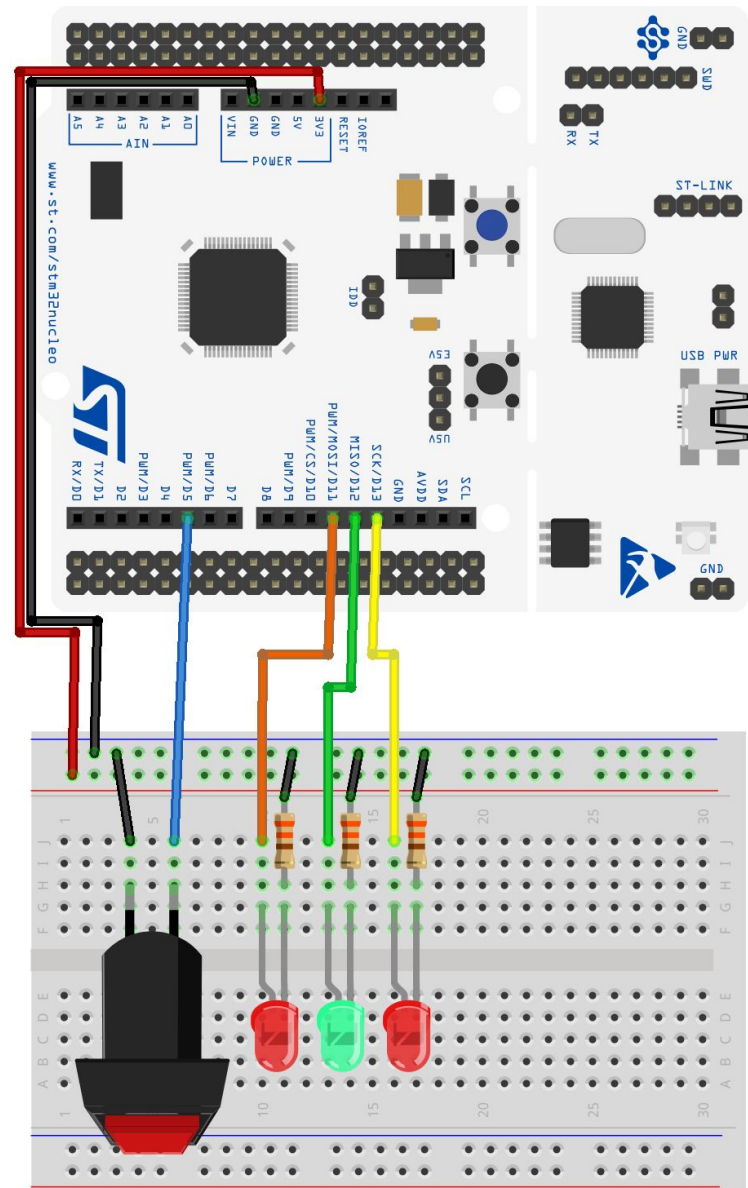


Pushbutton exercises

Make a copy of one of your Lab 1 programs and modify them to use the button:

- **blinkySEQ.c:** Pressing the button turns all the LEDs on, releasing it returns to the usual sequence.
- **blinkyCNT.c:** A button press causes the counter to pause, a second press causes it to resume.

(This is part of your Lab 2 work. You won't quite get this done right now but can get a nice headstart.)



“Polling” an input port

```
// MAIN EVENT LOOP
while(1) {
    if ((GPIOB->IDR & BUTTON_PIN) == 0) {
        GPIOA->ODR |= LED_PIN;
    } else {
        GPIOA->ODR &= ~LED_PIN;
    }
}
```

// Every time around the loop
// Check (“poll”) the pin
// Then take action

More general way: clearly separate reading the pin from acting:

```
uint8_t buttonPushed;
// MAIN EVENT LOOP
while(1) {
    buttonPushed = ((GPIOB->IDR & BUTTON_PIN) == 0);
    if (buttonPushed) {
        GPIOA->ODR |= LED_PIN;
    } else {
        GPIOA->ODR &= ~LED_PIN;
    }
}
```

// Every time around the loop
// Poll the pin
// Take action

Example: Using button to reset a counter

```
#include "ES28.h"                                // All the port definitions are here
#define GPIOBEN      (1U<<1)                    // Clock access to GPIOB (button)
#define BUTTON_PIN    (1U<<4)                    // Button on PB4
(...)

int main(void) {
    unsigned buttonPushed = 0;
    unsigned count = 0;

    RCC->IOPENR |= GPIOBEN;                        // Enable clock access to GPIOB

    GPIOB->MODER &= ~(1U<<8);                      // Configure PB4 as input pin
    GPIOB->MODER &= ~(1U<<9);                      // (not necessary since this is default reset value)

    GPIOB->PUPDR |= (1U<<8);                       // enable pull-up for PB4
    GPIOB->PUPDR &= ~(1U<<9);

    (...)
    while(1) {
        delay_ms(500);                             // kill some time
        buttonPushed = ((GPIOB->IDR & BUTTON_PIN) == 0); // Poll the button
        if ((buttonPushed) || (count == 7)) {        // Act on the button
            count = 0;                                // Reset the counter
        } else {
            count++;                                  // Increment the counter
        }
        /* do stuff with count, like turn on some LEDs */
    }
    return 0;
}
```

Now let's talk about the homework for Thursday

Does this work (once we've debounced the switch)?

```
while(1) {  
    buttonPushed = ((GPIOB->IDR & BUTTON_PIN) == 0); // Poll the button (1 => pressed)  
  
    if ( buttonPushed ) {                                // Act on the button  
  
        // toggle the LED                                // on-->off or off-->on  
  
    }  
    // implicit "do nothing" if button isn't pressed  
}
```

Give it a try. Think about the homework code to toggle the LED whenever the button is pushed and discuss at your table. What difficulties do you observe?

Often you need to sense both states of the switch

```
unsigned buttonAlreadyPressed = 0;

while(1) {
    buttonPushed = ((GPIOB->IDR & BUTTON_PIN) == 0); // Poll the button (1 => pressed)

    if (buttonAlreadyPressed==0) {                    // Has button been pressed?
        if ( buttonPushed==1 ) {                      // on-->off or off-->on
            // toggle the LED                          // Mark button as pressed
            buttonAlreadyPressed = 1;
        }
    } else {                                          // Button is still pressed
        if ( buttonPushed==0 ) {                      // Do nothing until button is released
            buttonAlreadyPressed = 0;                  // Mark button as released
        }
    }
}
```


On Thursday

More about watching buttons, in preparation for Lab 2.

State machines.